

## Simulador de Gestor de Procesos

Simulador de Gestor de Procesos

Dashboard

Configuracion

Ejecucion

Sincronizacion

Linea de tiempo

Comparacion

Historial

| PID | Nombre     | Estado | Prioridad | CPU Restante | CPU Asignada | Memoria Asignada |
|-----|------------|--------|-----------|--------------|--------------|------------------|
| 1   | Proceso 1  | NEW    | 1         | 6            | 0            | 0                |
| 2   | Proceso 2  | NEW    | 5         | 2            | 0            | 0                |
| 3   | Proceso 3  | NEW    | 10        | 3            | 0            | 0                |
| 4   | Proceso 4  | NEW    | 5         | 13           | 0            | 0                |
| 5   | Proceso 5  | NEW    | 10        | 20           | 0            | 0                |
| 6   | Proceso 6  | NEW    | 3         | 7            | 0            | 0                |
| 7   | Proceso 7  | NEW    | 10        | 16           | 0            | 0                |
| 8   | Proceso 8  | NEW    | 7         | 16           | 0            | 0                |
| 9   | Proceso 9  | READY  | 9         | 5            | 0            | 70               |
| 10  | Proceso 10 | READY  | 4         | 9            | 0            | 368              |
| 11  | Proceso 11 | NEW    | 6         | 11           | 0            | 0                |
| 12  | Proceso 12 | NEW    | 5         | 18           | 0            | 0                |
| 13  | Proceso 13 | NEW    | 4         | 2            | 0            | 0                |
| 14  | Proceso 14 | NEW    | 3         | 16           | 0            | 0                |
| 15  | Proceso 15 | READY  | 1         | 4            | 0            | 196              |

Mettricas actuales

Registro de eventos

[t=0] Proceso programado para llegada futura: PID 32  
[t=0] Proceso programado para llegada futura: PID 33  
[t=0] Proceso programado para llegada futura: PID 34  
[t=0] Proceso programado para llegada futura: PID 35  
[t=0] Proceso programado para llegada futura: PID 36  
[t=0] Proceso programado para llegada futura: PID 37  
[t=0] Proceso programado para llegada futura: PID 38  
[t=0] Proceso programado para llegada futura: PID 39  
[t=0] Proceso programado para llegada futura: PID 40

Listo para simular procesos.

Materia: Sistemas Operativos

Institución: Universidad Autónoma de Tamaulipas, Facultad de Ingeniería Tampico

Semestre: 2026-1

Profesor: Muñoz Quintero Dante Adolfo

Integrantes:

- Diaz de Leon Torres Bryan de Jesus
  - Tello Ponce Fabian Enrique
  - Ortiz Sosa Jose Jareth
- Gomez Sanchez Haziel Emiliano

Fecha: Mayo 2026

## Contenido

|                                            |                  |
|--------------------------------------------|------------------|
| <b>Introducción – .....</b>                | <b>página 3</b>  |
| <b>Objetivos –.....</b>                    | <b>página 3</b>  |
| <b>Diseño del sistema – .....</b>          | <b>página 4</b>  |
| <b>Algoritmos de planificación – .....</b> | <b>página 5</b>  |
| <b>Eventos estructurados – .....</b>       | <b>página 6</b>  |
| <b>Conclusiones –.....</b>                 | <b>página 30</b> |
| <b>Referencias –.....</b>                  | <b>página 30</b> |

## Introducción

### Contexto y motivación

Este proyecto desarrolla un simulador didáctico para entender y comparar algoritmos de planificación de procesos y mecanismos de sincronización en sistemas operativos. Está pensado como herramienta pedagógica que permite observar paso a paso la ejecución de procesos, la asignación de recursos, la aparición de bloqueos por sincronización y la recuperación / notificación entre productores y consumidores.

### Alcance

El simulador opera con un reloj discreto por unidades de tiempo, soporta múltiples algoritmos de planificación, modelos básicos de gestión de memoria y primitivas de sincronización (mutex y semáforos). Ofrece visualización tipo Gantt, gráficos de ráfagas, tablas de procesos y una demostración interactiva del problema productor-consumidor con buffer configurable.

## Objetivos

### Objetivo general

Construir una aplicación educativa que permita estudiar y comparar el comportamiento de distintos algoritmos de planificación y visualizar problemas de sincronización en ejecuciones reales de ejemplo.

### Objetivos específicos

- Implementar y comparar FCFS, SJF, Round Robin y Prioridad. ○  
Registrar eventos estructurados que permitan construir una línea de tiempo Gantt y marcar eventos de sincronización.
- Simular el problema productor-consumidor con mutex y semáforos, con buffer configurable ( $>1$ ).
- Mostrar recursos asignados por proceso (CPU y memoria) en la interfaz. ○  
Detectar y reportar interbloqueos (deadlocks) con heurística básica y pruebas de estrés para la demo.
- Proveer controles de demo (play/pause/step/reset) y vistas gráficas para análisis.

## Diseño del sistema

### Visión general

El sistema está organizado en módulos: una capa de modelo que representa procesos, recursos y estado global del simulador; servicios que implementan la lógica de planificación y sincronización; un controlador que actúa como puente entre la lógica y la presentación; y una interfaz gráfica que agrupa vistas y controles para el usuario.

### Componentes clave

- o Modelo de proceso: cada proceso mantiene su identificador, nombre, estado, prioridad, ráfaga de CPU total, CPU consumida, memoria requerida, tiempos acumulados y métricas.
- o Gestor de recursos: modelo centralizado de CPU y memoria que controla reservas y liberaciones.
- o Servicios de planificación: unidades que encapsulan la lógica paso a paso de cada algoritmo, emitiendo eventos descriptivos para la visualización.
- o Servicios de sincronización: primitivas de exclusión mutua y semáforos que mantienen colas de espera y permiten simular bloqueos y desbloques.
- o Vista: interfaz con pestañas para Dashboard, Configuración, Ejecución, Sincronización, Línea de tiempo, Comparación e Historial.

### Flujo de datos

La interacción típica es: el usuario activa una acción en la interfaz → el controlador la delega al servicio correspondiente → el servicio actualiza el estado global y registra eventos → la vista consulta el estado y renderiza tablas, gráficos y líneas de tiempo.

## 4. Implementación (resumen sin fragmentos de código)

### Metodología

El diseño prioriza separación de responsabilidades: la lógica de negocio (planificación y sincronización) reside en servicios independientes y el estado global permanece en una entidad accesible para la vista. Esto facilita pruebas, extensión y trazabilidad de eventos.

#### 4.1 Operaciones sobre procesos (PCB)

### Representación

Cada proceso incluye información necesaria para planificación y métricas: identificador, nombre, estado (nuevo, listo, ejecutando, esperando, terminado,

suspendido), prioridad, ráfaga total de CPU, CPU consumida, memoria requerida, tiempo de llegada, tiempos de espera y turnaround, y motivo de terminación.

## Funcionalidades

- Crear procesos con parámetros de prioridad, CPU y memoria. ○  
Ejecutar un proceso por una unidad de tiempo simulada.
- Suspender, reanudar y terminar procesos, con liberación controlada de recursos.
- Incremento automático de tiempos de espera para procesos en READY o WAITING.

## 4.2 ASIGNACIÓN DE RECURSOS

### Modelo de recursos

Un gestor central mantiene capacidad total de CPU y memoria, cantidad usada y disponibilidad. Las reservas y liberaciones se realizan de forma atómica a nivel lógico.

### Política

- Antes de admitir un proceso en READY se verifica la memoria solicitada; si hay capacidad, se reserva, si no, el proceso queda en WAITING.
- La CPU se asigna por unidad de tiempo durante la ejecución; cuando un proceso entra en RUNNING se verifica si existe CPU disponible y se consume una unidad, liberándola al terminar esa unidad.

### Visibilidad en la UI

La interfaz muestra por proceso cuánta CPU y memoria están asignadas actualmente, para ayudar a entender el consumo global y por proceso.

## 4.3 Algoritmos de planificación

### Implementación conceptual

Cada algoritmo (FCFS, SJF, Round Robin, Prioridad) está encapsulado en su propio servicio que:

- o Admite procesos elegibles. o Selecciona el próximo proceso según la política. o Registra eventos estructurados para marcar dispatch/start/end/preempt/finish. o Ejecuta un paso (una unidad de CPU o hasta quantum, en Round Robin).

## Eventos estructurados

La emisión de eventos estructurados permite construir la línea de tiempo: eventos como DISPATCH, START, END, PREEMPT, FINISH se utilizan para determinar intervalos de ejecución y dibujar rectángulos Gantt.

### 4.4 Comunicación y sincronización

#### Problema productor-consumidor

- o Buffer acotado con capacidad configurable.
- o Mutex para exclusión mutua sobre la sección crítica que accede al buffer.
  - o Dos semáforos: uno que cuenta espacios vacíos (empty) y otro que cuenta elementos disponibles (filled).

#### Comportamiento simulado

- o Un productor espera si no hay espacio ( $\text{empty}==0$ ), toma mutex, escribe, libera mutex y señala filled.
- o Un consumidor espera si no hay elementos ( $\text{filled}==0$ ), toma mutex, lee, libera mutex y señala empty.

#### Registro de eventos

La simulación registra intentos, bloqueos, entradas en sección crítica, escrituras/lecturas y señales de desbloqueo. Estos eventos se usan para marcar la línea de tiempo y para la visualización del estado de las colas de espera.

## 4.5 Terminación de procesos

#### Finalización centralizada

Cuando un proceso termina se marca su estado, se calculan métricas de turnaround, se liberan recursos reservados y se registra la terminación con motivo. La lógica de finalización está centralizada para consistencia.

### 4.6 Interfaz de usuario (sin capturas de código)

#### Pestañas principales

- Dashboard: tabla de procesos (PID, nombre, estado, prioridad, CPU restante, CPU asignada, memoria asignada), métricas actuales y registro de eventos.
- Configuración: creación/importación de procesos, selección de algoritmo.
  - Ejecución: controles para paso a paso y ejecución completa, tabla de detalle y gráfico de ráfagas.
- Sincronización: controles para producir/consumir manualmente, demo automático con pasos guiados y snapshot textual de mutex/semaforos/buffer.
  - Línea de tiempo: Gantt por proceso con marcadores de eventos de sincronización y una fila adicional que muestra la ocupación del buffer a lo largo del tiempo.
  - Comparación: gráfico con métricas comparadas (espera promedio, turnaround, uso de CPU) entre algoritmos.

Controles didácticos o Demo automatizada con  
 play/pause/step/reset. o Entrada de PIDs para pruebas  
 manuales de productor/consumidor.

- Campo para configurar tamaño del buffer en la demo.

## 5. Demostración funcional (qué capturar y qué observar)

- Creación de procesos:

## Simulador de Gestor de Procesos

Dashboard Configuración Ejecución Sincronización Línea de tiempo Comparación Historial

| PID | Nombre     | Estado | Prioridad | CPU Restante | CPU Asignada | Memoria Asignada |
|-----|------------|--------|-----------|--------------|--------------|------------------|
| 1   | Proceso 1  | NEW    | 8         | 7            | 0            | 0                |
| 2   | Proceso 2  | NEW    | 10        | 17           | 0            | 0                |
| 3   | Proceso 3  | NEW    | 1         | 4            | 0            | 0                |
| 4   | Proceso 1  | NEW    | 9         | 15           | 0            | 0                |
| 5   | Proceso 2  | READY  | 2         | 5            | 0            | 151              |
| 6   | Proceso 3  | NEW    | 6         | 19           | 0            | 0                |
| 7   | Proceso 4  | NEW    | 6         | 2            | 0            | 0                |
| 8   | Proceso 5  | NEW    | 4         | 15           | 0            | 0                |
| 9   | Proceso 6  | NEW    | 6         | 1            | 0            | 0                |
| 10  | Proceso 7  | NEW    | 6         | 15           | 0            | 0                |
| 11  | Proceso 8  | NEW    | 5         | 3            | 0            | 0                |
| 12  | Proceso 9  | READY  | 3         | 3            | 0            | 557              |
| 13  | Proceso 10 | NEW    | 8         | 2            | 0            | 0                |
| 14  | Proceso 11 | NEW    | 8         | 7            | 0            | 0                |
| 15  | Proceso 12 | NEW    | 2         | 11           | 0            | 0                |

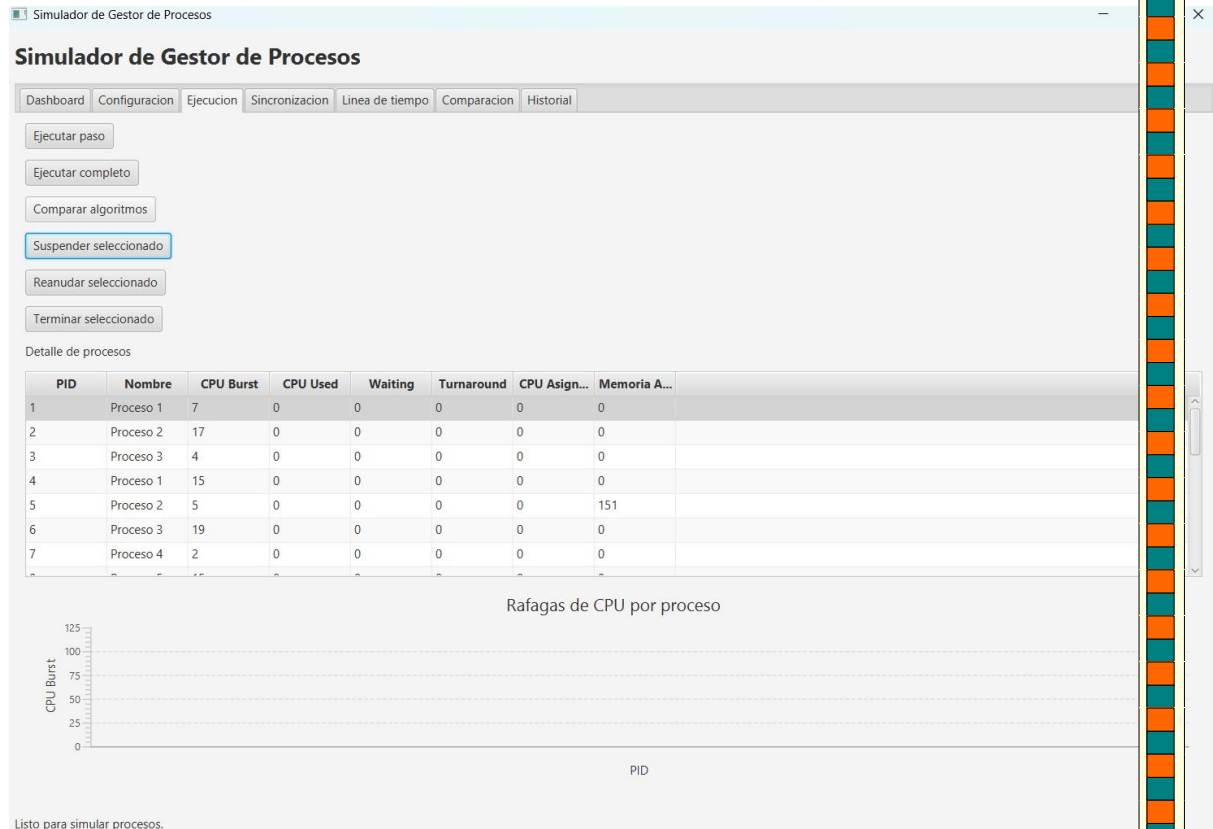
Métricas actuales

Registro de eventos

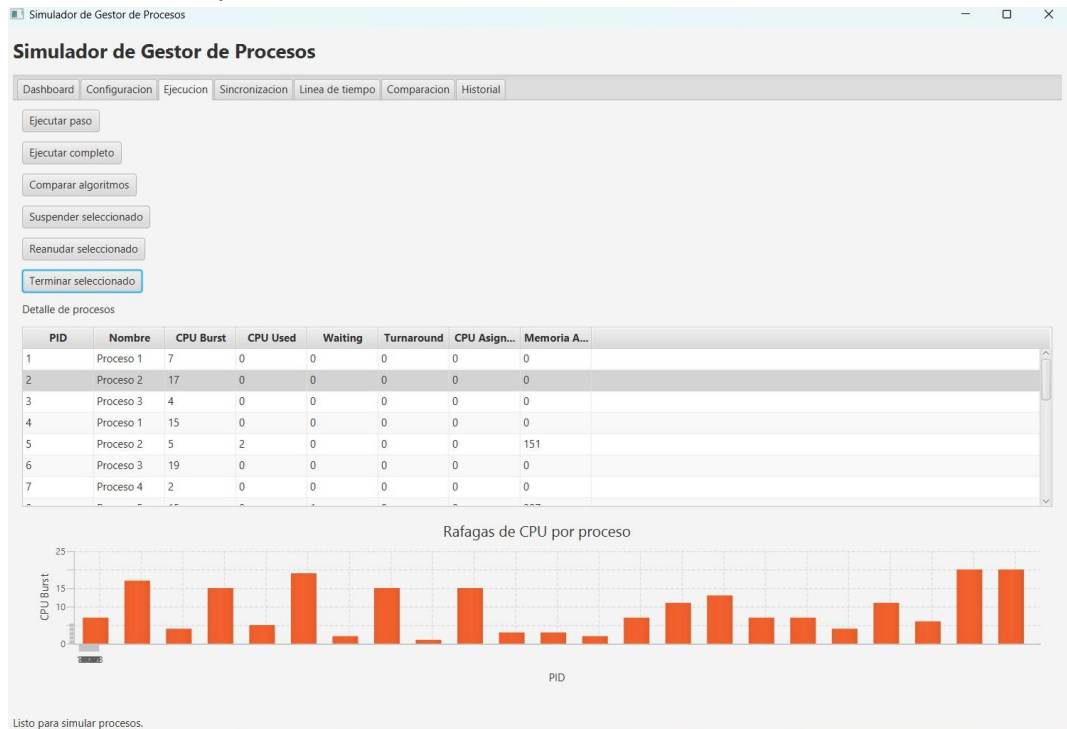
[t=0] Planificador activo: FCFS  
[t=0] Proceso programado para llegada futura: PID 1  
[t=0] Proceso programado para llegada futura: PID 2  
[t=0] Proceso programado para llegada futura: PID 3  
[t=0] Planificador activo: FCFS  
[t=0] Planificador activo: FCFS  
[t=0] Planificador activo: Round Robin  
[t=0] Proceso programado para llegada futura: PID 4  
[t=0] Proceso admitido a READY: PID 5

Listo para simular procesos.

- Suspension de procesos:



- Terminacion de procesos:



Planificación con cada algoritmo:

- FCFS: CODIGO --- FcfsExecutionService.java ---
- package com.os.simulator.services;
- 
- import com.os.simulator.model.Process;
- import com.os.simulator.model.ProcessState;
- import com.os.simulator.model.SystemState;
- 
- /\*\*
- \* Ejecuta la logica de FCFS paso a paso.
- \*/
- public class FcfsExecutionService implements

AlgorithmExecutionService {

- @Override
- public Process executeStep(SimulationService simulationService) {
- // FCFS favorece el orden de llegada, asi que no reordena la cola.
- SystemState state = simulationService.getSystemState();
- state.tryAdmitEligibleProcesses();
- 
- if (state.getReadyQueue().isEmpty()) {
- state.logEvent("No hay procesos en cola de listos para FCFS.");
- state.advanceTime();
- simulationService.clearLastDispatchedProcess();
- return null;
- }
- 
- Process process =

simulationService.getScheduler().selectNextProcess(state.getReadyQueue(

));

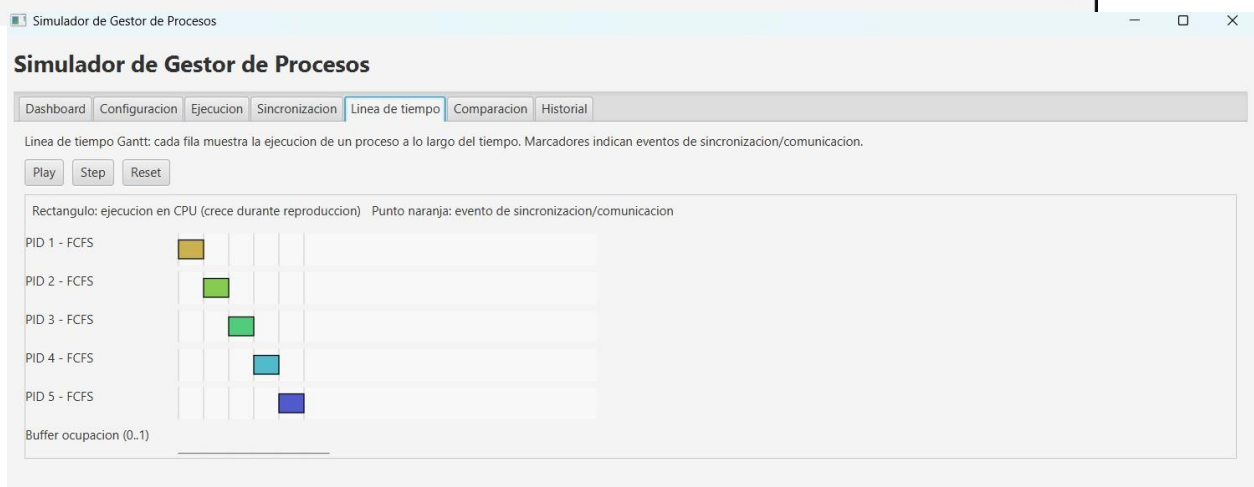
- if (process == null) {
- state.logEvent("FCFS no retorno un proceso valido.");
- state.advanceTime();
- simulationService.clearLastDispatchedProcess();
- return null;
- }
- 
- simulationService.registerContextSwitch(process);
- process.setState(ProcessState.RUNNING);
- state.logEvent("FCFS ejecuta PID " + process.getPid());

- // Structured events for UI timeline
- state.logEvent("EVENT:DISPATCH PID " + process.getPid());
- state.logEvent("EVENT:START PID " + process.getPid());
- 
- if (!state.getResource().allocate(1, 0)) {
- // Si no hay CPU disponible, el proceso vuelve a READY para no perder su lugar logico.
- state.logEvent("FCFS no pudo asignar CPU al PID " + process.getPid());
- process.setState(ProcessState.READY);
- state.enqueueReadyProcess(process);
- state.advanceTime();
- simulationService.clearLastDispatchedProcess();
- return process;
- }
- 
- // registrar unidad de CPU asignada para visibilidad en UI
- process.setAllocatedCpuUnits(process.getAllocatedCpuUnits() + 1);
- 
- process.executeOneUnit();
- simulationService.incrementExecutedCpuUnits(1);
- state.getResource().release(1, 0);
- process.setAllocatedCpuUnits(Math.max(0, process.getAllocatedCpuUnits() - 1));
- state.advanceTime();
- 
- if (process.isFinished()) {
- // Cuando termina se libera memoria y se registra la causa de cierre.
- simulationService.finishProcess(process, "Finalizado normalmente");
- state.logEvent("EVENT:END PID " + process.getPid());
- state.logEvent("EVENT:FINISH PID " + process.getPid());
- } else {
- // Si aun no termina, se vuelve a colocar al final de la cola para mantener el comportamiento de cola.
- process.setState(ProcessState.READY);

```

•         state.enqueueReadyProcess(process);
•         state.logEvent("FCFS devuelve PID " + process.getPid() + " al
final de READY");
•         state.logEvent("EVENT:END PID " + process.getPid());
•         state.logEvent("EVENT:PREEMPT PID " + process.getPid());
•     }
•
•         simulationService.setLastDispatchedProcess(process);
•         return process;
•     }
•
•     @Override
•     public String getName() {
•         return "FCFS";
•     }
• }
•
• --- FCFSScheduler.java ---
• package com.os.simulator.services;
•
• import com.os.simulator.model.Process;
• import java.util.Queue;
•
• /**
•  * Algoritmo FCFS: selecciona el primer proceso que llego a la cola.
•  */
• public class FCFSScheduler implements Scheduler {
•     @Override
•     public Process selectNextProcess(Queue<Process> readyQueue) {
•         // FCFS respeta estrictamente el orden de llegada, por eso solo
extrae el primero.
•         return readyQueue.poll();
•     }
•
•     @Override
•     public String getName() {
•         return "FCFS";
•     }

```

[illegible]

SFJ: CODIGO SFJ

```
/**
 * Algoritmo SJF: selecciona el proceso con menor rafaga restante de CPU.
 */
public class SJFScheduler implements Scheduler {
    @Override
    public Process selectNextProcess(Queue<Process> readyQueue) {
        // SJF busca el proceso con menor CPU restante para reducir el tiempo
        // promedio de espera.
        Process selected = readyQueue.stream()
            .min(Comparator.comparingInt(Process::getRemainingCpu))
            .orElse(null);

        if (selected != null) {
            readyQueue.remove(selected);
        }

        return selected;
    }

    @Override
    public String getName() {
        return "SJF";
    }
}
```

Round Robin

```
--- RoundRobinExecutionService.java ---
package com.os.simulator.services;

import com.os.simulator.model.Process;
import com.os.simulator.model.ProcessState;
import com.os.simulator.model.SystemState;
```

```

/**
 * Ejecuta la logica de Round Robin paso a paso usando el quantum
 configurado.
 */
public class RoundRobinExecutionService implements
AlgorithmExecutionService {
    @Override
    public Process executeStep(SimulationService simulationService) {
        // Round Robin reparte CPU por turnos fijos para evitar que un solo
 proceso acapare el procesador.
        SystemState state = simulationService.getSystemState();
        state.tryAdmitEligibleProcesses();

        if (state.getReadyQueue().isEmpty()) {
            state.logEvent("No hay procesos en cola de listos para Round Robin.");
            state.advanceTime();
            simulationService.clearLastDispatchedProcess();
            return null;
        }

        Process process =
simulationService.getScheduler().selectNextProcess(state.getReadyQueue(
));
        if (process == null) {
            state.logEvent("Round Robin no retorno un proceso valido.");
            state.advanceTime();
            simulationService.clearLastDispatchedProcess();
            return null;
        }

        simulationService.registerContextSwitch(process);
        process.setState(ProcessState.RUNNING);
        state.logEvent("Round Robin ejecuta PID " + process.getPid());
        state.logEvent("EVENT:DISPATCH PID " + process.getPid());
        state.logEvent("EVENT:START PID " + process.getPid());

        int quantum = simulationService.getRoundRobinQuantum();

```

```

// Se ejecuta hasta agotar el quantum o hasta que el proceso termine.
int executedUnits = 0;
while (executedUnits < quantum && !process.isFinished()) {
    if (!state.getResource().allocate(1, 0)) {
        state.logEvent("Round Robin no pudo asignar CPU al PID " +
process.getPid());
        break;
    }
    // marcar CPU asignada para UI
    process.setAllocatedCpuUnits(process.getAllocatedCpuUnits() + 1);

    process.executeOneUnit();
    simulationService.incrementExecutedCpuUnits(1);
    state.getResource().release(1, 0);
    process.setAllocatedCpuUnits(Math.max(0,
process.getAllocatedCpuUnits() - 1));
    state.advanceTime();
    executedUnits++;
}

if (process.isFinished()) {
    // Si termina dentro de su turno, se registra como concluido de
inmediato.
    simulationService.finishProcess(process, "Finalizado normalmente");
    state.logEvent("EVENT:END PID " + process.getPid());
    state.logEvent("EVENT:FINISH PID " + process.getPid());
} else {
    // Si no termina, vuelve al final de la cola para que otros procesos
tengan oportunidad.
    process.setState(ProcessState.READY);
    state.enqueueReadyProcess(process);
    state.logEvent("Round Robin devuelve PID " + process.getPid() + " al
final de READY");
    state.logEvent("EVENT:END PID " + process.getPid());
    state.logEvent("EVENT:PREEMPT PID " + process.getPid());
}

simulationService.setLastDispatchedProcess(process);

```

```
        return process;
    }

    @Override
    public String getName() {
        return "Round Robin";
    }
}
```

## Simulador de Gestor de Procesos

Dashboard Configuración Ejecución Sincronización Línea de tiempo Comparación Historial

| PID | Nombre | Estado     | Prioridad | CPU Restante | CPU Asignada | Memoria Asignada |
|-----|--------|------------|-----------|--------------|--------------|------------------|
| 1   | FCFS   | TERMINATED | 5         | 0            | 0            | 0                |
| 2   | FCFS   | TERMINATED | 5         | 0            | 0            | 0                |
| 3   | FCFS   | TERMINATED | 5         | 0            | 0            | 0                |
| 4   | FCFS   | TERMINATED | 5         | 0            | 0            | 0                |
| 5   | FCFS   | TERMINATED | 5         | 0            | 0            | 0                |
| 6   | SJF    | READY      | 5         | 1            | 0            | 4092             |
| 7   | SJF    | WAITING    | 5         | 1            | 0            | 0                |
| 8   | SJF    | WAITING    | 5         | 1            | 0            | 0                |
| 9   | SJF    | WAITING    | 5         | 1            | 0            | 0                |
| 10  | SJF    | WAITING    | 5         | 1            | 0            | 0                |
| 11  | SJF    | WAITING    | 5         | 1            | 0            | 0                |
|     |        |            |           |              |              |                  |
|     |        |            |           |              |              |                  |
|     |        |            |           |              |              |                  |

### Métricas actuales

Procesos: 5 | Terminados: 5 | Espera promedio: 2.00 | Retorno promedio: 3.00 | Cambios de contexto: 4 | CPU: 100.00% | Throughput: 1.0000

Recursos: CPU total: 4 núcleos | CPU usadas: 0 | Memoria total: 4096 MB | Memoria usada: 0 MB

### Registro de eventos

[t=5] Proceso terminado: PID 5 - Finalizado normalmente  
[t=5] EVENT:END PID 5  
[t=5] EVENT:FINISH PID 5  
[t=5] Proceso admitido a READY: PID 6  
[t=5] Proceso en espera por memoria: PID 7  
[t=5] Proceso en espera por memoria: PID 8  
[t=5] Proceso en espera por memoria: PID 9  
[t=5] Proceso en espera por memoria: PID 10  
[t=5] Proceso en espera por memoria: PID 11

Listo para simular procesos.

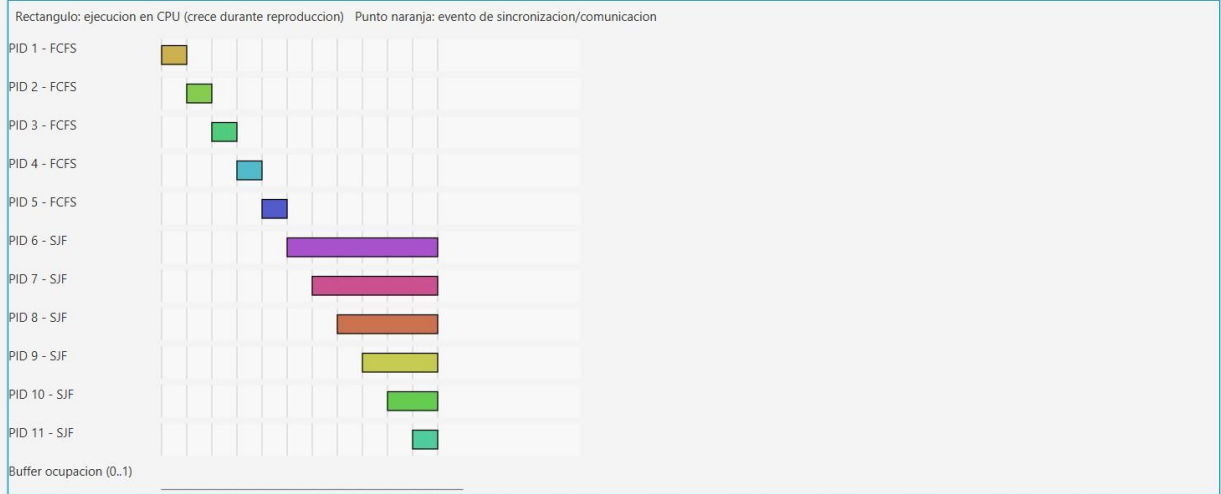
Simulador de Gestor de Procesos

## Simulador de Gestor de Procesos

Dashboard Configuración Ejecución Sincronización Línea de tiempo Comparación Historial

Línea de tiempo Gantt: cada fila muestra la ejecución de un proceso a lo largo del tiempo. Marcadores indican eventos de sincronización/comunicación.

Play Step Reset



Listo para simular procesos.

## Round Robin: CODIGO

```
--- RoundRobinScheduler.java ---
package com.os.simulator.services;

import com.os.simulator.model.Process;
import java.util.Queue;

/**
 * Algoritmo Round Robin: ejecuta por quantum y devuelve el proceso al final
 * si no termino.
 */
public class RoundRobinScheduler implements Scheduler {

    // Quantum fijo para controlar cuantos turnos seguidos puede consumir
    // un proceso.

    private final int quantum;

    /**
     * Crea el planificador con un quantum definido.
     *
     * @param quantum unidades de tiempo por turno.
     */
    public RoundRobinScheduler(int quantum) {

        // Un quantum menor que uno no tendria sentido dentro de la
        // simulacion.

        this.quantum = quantum;
    }
}
```

```
public int getQuantum() {  
    return quantum;  
}
```

```
@Override
```

```
public Process selectNextProcess(Queue<Process> readyQueue) {  
    // Round Robin solo toma el siguiente proceso de la cola; el tiempo lo  
    controla el service de ejecucion.  
    return readyQueue.poll();  
}
```

```
@Override
```

```
public String getName() {  
    return "Round Robin";  
}  
}
```

Prioridad: CODIGO

```
-- PriorityExecutionService.java ---
package com.os.simulator.services;
```

```
import com.os.simulator.model.Process;
import com.os.simulator.model.ProcessState;
import com.os.simulator.model.SystemState;
```

|    |             |            |   |   |   |      |
|----|-------------|------------|---|---|---|------|
| 2  | FCFS        | TERMINATED | 5 | 0 | 0 | 0    |
| 3  | FCFS        | TERMINATED | 5 | 0 | 0 | 0    |
| 4  | FCFS        | TERMINATED | 5 | 0 | 0 | 0    |
| 5  | FCFS        | TERMINATED | 5 | 0 | 0 | 0    |
| 6  | SJF         | TERMINATED | 5 | 0 | 0 | 0    |
| 7  | SJF         | TERMINATED | 5 | 0 | 0 | 0    |
| 8  | SJF         | TERMINATED | 5 | 0 | 0 | 0    |
| 9  | SJF         | TERMINATED | 5 | 0 | 0 | 0    |
| 10 | SJF         | TERMINATED | 5 | 0 | 0 | 0    |
| 11 | SJF         | TERMINATED | 5 | 0 | 0 | 0    |
| 12 | Round Robin | READY      | 5 | 1 | 0 | 4092 |
| 13 | Round Robin | WAITING    | 5 | 1 | 0 | 0    |
| 14 | Round Robin | WAITING    | 5 | 1 | 0 | 0    |
| 15 | Round Robin | WAITING    | 5 | 1 | 0 | 0    |
| 16 | Round Robin | WAITING    | 5 | 1 | 0 | 0    |

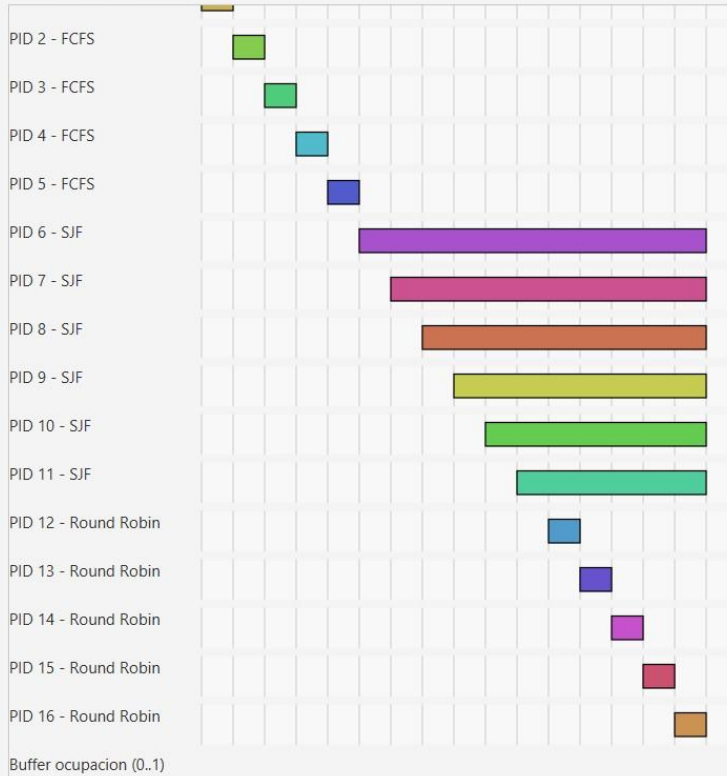
Simulador de Gestor de Procesos

## Simulador de Gestor de Procesos

Dashboard Configuracion Ejecucion Sincronizacion **Linea de tiempo** Comparacion Historial

Linea de tiempo Gantt: cada fila muestra la ejecucion de un proceso a lo largo del tiempo. Marcadores indican eventos de sincronizacion/comunicacion.

Play Step Reset



```

/**
 * Ejecuta la logica del algoritmo por prioridad paso a paso.
 */
public class PriorityExecutionService implements
AlgorithmExecutionService {
    @Override
    public Process executeStep(SimulationService simulationService) {
        // El criterio dominante es la prioridad: el numero mas alto debe
ejecutarse primero.
        SystemState state = simulationService.getSystemState();
        state.tryAdmitEligibleProcesses();

        if (state.getReadyQueue().isEmpty()) {
            state.logEvent("No hay procesos en cola de listos para Prioridad.");
            state.advanceTime();
            simulationService.clearLastDispatchedProcess();
            return null;
        }

        Process process =
simulationService.getScheduler().selectNextProcess(state.getReadyQueue(
));
        if (process == null) {
            state.logEvent("Prioridad no retorno un proceso valido.");
            state.advanceTime();
            simulationService.clearLastDispatchedProcess();
            return null;
        }

        simulationService.registerContextSwitch(process);
        process.setState(ProcessState.RUNNING);
        state.logEvent("Prioridad ejecuta PID " + process.getPid());
        state.logEvent("EVENT:DISPATCH PID " + process.getPid());
        state.logEvent("EVENT:START PID " + process.getPid());

        if (!state.getResource().allocate(1, 0)) {
            // Si no hay CPU disponible, el proceso no pierde su prioridad ni su
turno logico.

```

```

        state.logEvent("Prioridad no pudo asignar CPU al PID " +
process.getPid());
        process.setState(ProcessState.READY);
        state.enqueueReadyProcess(process);
        state.advanceTime();
        simulationService.clearLastDispatchedProcess();
        return process;
    }
    // marcar CPU asignada para UI
    process.setAllocatedCpuUnits(process.getAllocatedCpuUnits() + 1);

    process.executeOneUnit();
    simulationService.incrementExecutedCpuUnits(1);
    state.getResource().release(1, 0);
    process.setAllocatedCpuUnits(Math.max(0,
process.getAllocatedCpuUnits() - 1));
    state.advanceTime();

    if (process.isFinished()) {
        // Al terminar se cierra su ciclo de vida y libera los recursos reservados.
        simulationService.finishProcess(process, "Finalizado normalmente");
        state.logEvent("EVENT:END PID " + process.getPid());
        state.logEvent("EVENT:FINISH PID " + process.getPid());
    } else {
        // Si sigue activo, conserva su prioridad para competir nuevamente en
la siguiente seleccion.
        process.setState(ProcessState.READY);
        state.enqueueReadyProcess(process);
        state.logEvent("Prioridad devuelve PID " + process.getPid() + " a
READY");
        state.logEvent("EVENT:END PID " + process.getPid());
        state.logEvent("EVENT:PREEMPT PID " + process.getPid());
    }

    simulationService.setLastDispatchedProcess(process);
    return process;
}

@Override
public String getName() {

```

```

        return "Prioridad";
    }
}

--- PriorityScheduler.java ---
package com.os.simulator.services;

import com.os.simulator.model.Process;
import java.util.Comparator;
import java.util.Queue;

/**
 * Algoritmo por prioridades: elige el proceso con prioridad mas alta.
 */
public class PriorityScheduler implements Scheduler {
    @Override
    public Process selectNextProcess(Queue<Process> readyQueue) {
        // Se elige la prioridad mas alta porque ese es el criterio visible para el
        // usuario en este algoritmo.
        Process selected = readyQueue.stream()
            .max(Comparator.comparingInt(Process::getPriority))
            .orElse(null);
        if (selected != null) {
            readyQueue.remove(selected);
        }
        return selected;
    }
    @Override
    public String getName() {
        return "Prioridad";
    }
}

```

## Simulador de Gestor de Procesos

Dashboard Configuración Ejecución Sincronización Línea de tiempo Comparación Historial

| PID | Nombre      | Estado     | Prioridad | CPU Restante | CPU Asignada | Memoria Asignada |
|-----|-------------|------------|-----------|--------------|--------------|------------------|
| 7   | SJF         | TERMINATED | 5         | 0            | 0            | 0                |
| 8   | SJF         | TERMINATED | 5         | 0            | 0            | 0                |
| 9   | SJF         | TERMINATED | 5         | 0            | 0            | 0                |
| 10  | SJF         | TERMINATED | 5         | 0            | 0            | 0                |
| 11  | SJF         | TERMINATED | 5         | 0            | 0            | 0                |
| 12  | Round Robin | TERMINATED | 5         | 0            | 0            | 0                |
| 13  | Round Robin | TERMINATED | 5         | 0            | 0            | 0                |
| 14  | Round Robin | TERMINATED | 5         | 0            | 0            | 0                |
| 15  | Round Robin | TERMINATED | 5         | 0            | 0            | 0                |
| 16  | Round Robin | TERMINATED | 5         | 0            | 0            | 0                |
| 17  | Proceso 1   | TERMINATED | 10        | 0            | 0            | 0                |
| 18  | Proceso 2   | TERMINATED | 1         | 0            | 0            | 0                |
| 19  | Proceso 3   | TERMINATED | 6         | 0            | 0            | 0                |
| 20  | Proceso 4   | TERMINATED | 3         | 0            | 0            | 0                |
| 21  | Proceso 5   | TERMINATED | 9         | 0            | 0            | 0                |

### Métricas actuales

Procesos: 21 | Terminados: 21 | Espera promedio: 6.05 | Retorno promedio: 11.14 | Cambios de contexto: 20 | CPU: 100.00% | Throughput: 0.4200

Recursos: CPU total: 4 núcleos | CPU usadas: 0 | Memoria total: 4096 MB | Memoria usada: 0 MB

### Registro de eventos

[t=0] Planificador activo: FCFS  
 [t=0] Planificador activo: FCFS  
 [t=0] Proceso admitido a READY: PID 1  
 [t=0] Proceso en espera por memoria: PID 2  
 [t=0] Proceso en espera por memoria: PID 3  
 [t=0] Proceso en espera por memoria: PID 4  
 [t=0] Proceso en espera por memoria: PID 5  
 [t=0] Proceso en espera por memoria: PID 2  
 [t=0] Proceso en espera por memoria: PID 3

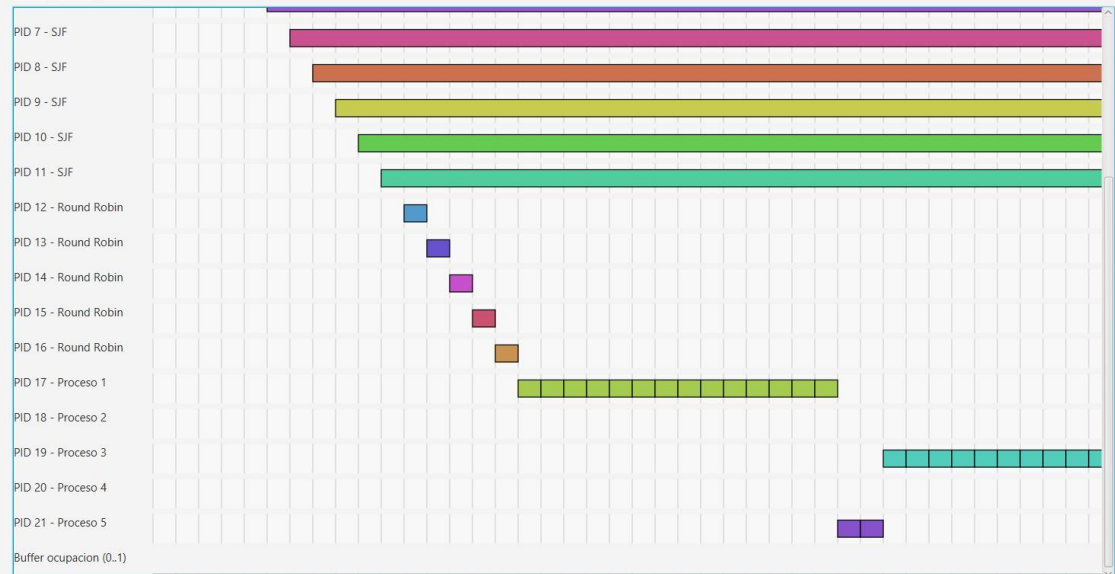
Listo para simular procesos.

## Simulador de Gestor de Procesos

Dashboard Configuración Ejecución Sincronización Línea de tiempo Comparación Historial

Línea de tiempo Gantt: cada fila muestra la ejecución de un proceso a lo largo del tiempo. Marcadores indican eventos de sincronización/comunicación.

Play Step Reset



Listo para simular procesos.

Mensajes de Log de algoritmos: [t=0] Planificador activo: FCFS

[t=0] Planificador activo: FCFS

[t=0] Proceso admitido a READY: PID 1

[t=0] Proceso en espera por memoria: PID 2

[t=0] Proceso en espera por memoria: PID 3

[t=0] Proceso en espera por memoria: PID 4

[t=0] Proceso en espera por memoria: PID 5

[t=0] Proceso en espera por memoria: PID 2

[t=0] Proceso en espera por memoria: PID 3

[t=0] Proceso en espera por memoria: PID 4

[t=0] Proceso en espera por memoria: PID 5

[t=0] FCFS ejecuta PID 1

[t=0] EVENT:DISPATCH PID 1

[t=0] EVENT:START PID 1

[t=1] Proceso en espera por memoria: PID 2

[t=1] Proceso en espera por memoria: PID 3

[t=1] Proceso en espera por memoria: PID 4

[t=1] Proceso en espera por memoria: PID 5

[t=1] Proceso terminado: PID 1 - Finalizado normalmente

[t=1] EVENT:END PID 1

[t=1] EVENT:FINISH PID 1

[t=1] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=1] Proceso admitido a READY: PID 2

[t=1] Proceso en espera por memoria: PID 3

[t=1] Proceso en espera por memoria: PID 4

[t=1] Proceso en espera por memoria: PID 5

[t=1] FCFS ejecuta PID 2

[t=1] EVENT:DISPATCH PID 2

[t=1] EVENT:START PID 2

[t=2] Proceso en espera por memoria: PID 3

[t=2] Proceso en espera por memoria: PID 4

[t=2] Proceso en espera por memoria: PID 5

[t=2] Proceso terminado: PID 2 - Finalizado normalmente

[t=2] EVENT:END PID 2

[t=2] EVENT:FINISH PID 2

[t=2] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=2] Proceso admitido a READY: PID 3

[t=2] Proceso en espera por memoria: PID 4

[t=2] Proceso en espera por memoria: PID 5

[t=2] FCFS ejecuta PID 3

[t=2] EVENT:DISPATCH PID 3

[t=2] EVENT:START PID 3

[t=3] Proceso en espera por memoria: PID 4

[t=3] Proceso en espera por memoria: PID 5

[t=3] Proceso terminado: PID 3 - Finalizado normalmente

[t=3] EVENT:END PID 3

[t=3] EVENT:FINISH PID 3

[t=3] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=3] Proceso admitido a READY: PID 4

[t=3] Proceso en espera por memoria: PID 5

[t=3] FCFS ejecuta PID 4

[t=3] EVENT:DISPATCH PID 4

[t=3] EVENT:START PID 4

[t=4] Proceso en espera por memoria: PID 5

[t=4] Proceso terminado: PID 4 - Finalizado normalmente

[t=4] EVENT:END PID 4

[t=4] EVENT:FINISH PID 4

[t=4] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=4] Proceso admitido a READY: PID 5

[t=4] FCFS ejecuta PID 5

[t=4] EVENT:DISPATCH PID 5

[t=4] EVENT:START PID 5

[t=5] Proceso terminado: PID 5 - Finalizado normalmente

[t=5] EVENT:END PID 5

[t=5] EVENT:FINISH PID 5

[t=5] Proceso admitido a READY: PID 6

[t=5] Proceso en espera por memoria: PID 7

[t=5] Proceso en espera por memoria: PID 8

[t=5] Proceso en espera por memoria: PID 9

[t=5] Proceso en espera por memoria: PID 10

[t=5] Proceso en espera por memoria: PID 11

[t=5] Planificador activo: SJF

[t=5] Proceso en espera por memoria: PID 7

[t=5] Proceso en espera por memoria: PID 8

[t=5] Proceso en espera por memoria: PID 9

[t=5] Proceso en espera por memoria: PID 10

[t=5] Proceso en espera por memoria: PID 11

[t=5] SJF ejecuta PID 6 con menor rafaga restante

[t=5] EVENT:DISPATCH PID 6

[t=5] EVENT:START PID 6

[t=6] Proceso en espera por memoria: PID 7

[t=6] Proceso en espera por memoria: PID 8

[t=6] Proceso en espera por memoria: PID 9  
[t=6] Proceso en espera por memoria: PID 10  
[t=6] Proceso en espera por memoria: PID 11  
[t=6] Proceso terminado: PID 6 - Finalizado normalmente  
[t=6] EVENT:END PID 6  
[t=6] DEADLOCK DETECTED durante ejecucion de algoritmo.  
[t=6] Proceso admitido a READY: PID 7  
[t=6] Proceso en espera por memoria: PID 8  
[t=6] Proceso en espera por memoria: PID 9  
[t=6] Proceso en espera por memoria: PID 10  
[t=6] Proceso en espera por memoria: PID 11  
[t=6] SJF ejecuta PID 7 con menor rafaga restante  
[t=6] EVENT:DISPATCH PID 7  
[t=6] EVENT:START PID 7  
[t=7] Proceso en espera por memoria: PID 8  
[t=7] Proceso en espera por memoria: PID 9  
[t=7] Proceso en espera por memoria: PID 10  
[t=7] Proceso en espera por memoria: PID 11  
[t=7] Proceso terminado: PID 7 - Finalizado normalmente  
[t=7] EVENT:END PID 7  
[t=7] DEADLOCK DETECTED durante ejecucion de algoritmo.  
[t=7] Proceso admitido a READY: PID 8  
[t=7] Proceso en espera por memoria: PID 9  
[t=7] Proceso en espera por memoria: PID 10  
[t=7] Proceso en espera por memoria: PID 11  
[t=7] SJF ejecuta PID 8 con menor rafaga restante  
[t=7] EVENT:DISPATCH PID 8

[t=7] EVENT:START PID 8

[t=8] Proceso en espera por memoria: PID 9

[t=8] Proceso en espera por memoria: PID 10

[t=8] Proceso en espera por memoria: PID 11

[t=8] Proceso terminado: PID 8 - Finalizado normalmente

[t=8] EVENT:END PID 8

[t=8] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=8] Proceso admitido a READY: PID 9

[t=8] Proceso en espera por memoria: PID 10

[t=8] Proceso en espera por memoria: PID 11

[t=8] SJF ejecuta PID 9 con menor rafaga restante

[t=8] EVENT:DISPATCH PID 9

[t=8] EVENT:START PID 9

[t=9] Proceso en espera por memoria: PID 10

[t=9] Proceso en espera por memoria: PID 11

[t=9] Proceso terminado: PID 9 - Finalizado normalmente

[t=9] EVENT:END PID 9

[t=9] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=9] Proceso admitido a READY: PID 10

[t=9] Proceso en espera por memoria: PID 11

[t=9] SJF ejecuta PID 10 con menor rafaga restante

[t=9] EVENT:DISPATCH PID 10

[t=9] EVENT:START PID 10

[t=10] Proceso en espera por memoria: PID 11

[t=10] Proceso terminado: PID 10 - Finalizado normalmente

[t=10] EVENT:END PID 10

[t=10] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=10] Proceso admitido a READY: PID 11

[t=10] SJF ejecuta PID 11 con menor rafaga restante

[t=10] EVENT:DISPATCH PID 11

[t=10] EVENT:START PID 11

[t=11] Proceso terminado: PID 11 - Finalizado normalmente

[t=11] EVENT:END PID 11

[t=11] Proceso admitido a READY: PID 12

[t=11] Proceso en espera por memoria: PID 13

[t=11] Proceso en espera por memoria: PID 14

[t=11] Proceso en espera por memoria: PID 15

[t=11] Proceso en espera por memoria: PID 16

[t=11] Planificador activo: Round Robin

[t=11] Proceso en espera por memoria: PID 13

[t=11] Proceso en espera por memoria: PID 14

[t=11] Proceso en espera por memoria: PID 15

[t=11] Proceso en espera por memoria: PID 16

[t=11] Round Robin ejecuta PID 12

[t=11] EVENT:DISPATCH PID 12

[t=11] EVENT:START PID 12

[t=12] Proceso en espera por memoria: PID 13

[t=12] Proceso en espera por memoria: PID 14

[t=12] Proceso en espera por memoria: PID 15

[t=12] Proceso en espera por memoria: PID 16

[t=12] Proceso terminado: PID 12 - Finalizado normalmente

[t=12] EVENT:END PID 12

[t=12] EVENT:FINISH PID 12

[t=12] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=12] Proceso admitido a READY: PID 13

[t=12] Proceso en espera por memoria: PID 14

[t=12] Proceso en espera por memoria: PID 15

[t=12] Proceso en espera por memoria: PID 16

[t=12] Round Robin ejecuta PID 13

[t=12] EVENT:DISPATCH PID 13

[t=12] EVENT:START PID 13

[t=13] Proceso en espera por memoria: PID 14

[t=13] Proceso en espera por memoria: PID 15

[t=13] Proceso en espera por memoria: PID 16

[t=13] Proceso terminado: PID 13 - Finalizado normalmente

[t=13] EVENT:END PID 13

[t=13] EVENT:FINISH PID 13

[t=13] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=13] Proceso admitido a READY: PID 14

[t=13] Proceso en espera por memoria: PID 15

[t=13] Proceso en espera por memoria: PID 16

[t=13] Round Robin ejecuta PID 14

[t=13] EVENT:DISPATCH PID 14

[t=13] EVENT:START PID 14

[t=14] Proceso en espera por memoria: PID 15

[t=14] Proceso en espera por memoria: PID 16

[t=14] Proceso terminado: PID 14 - Finalizado normalmente

[t=14] EVENT:END PID 14

[t=14] EVENT:FINISH PID 14

[t=14] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=14] Proceso admitido a READY: PID 15

[t=14] Proceso en espera por memoria: PID 16

[t=14] Round Robin ejecuta PID 15

[t=14] EVENT:DISPATCH PID 15

[t=14] EVENT:START PID 15

[t=15] Proceso en espera por memoria: PID 16

[t=15] Proceso terminado: PID 15 - Finalizado normalmente [t=15] EVENT:END PID 15

[t=15] EVENT:FINISH PID 15

[t=15] DEADLOCK DETECTED durante ejecucion de algoritmo.

[t=15] Proceso admitido a READY: PID 16

[t=15] Round Robin ejecuta PID 16

[t=15] EVENT:DISPATCH PID 16

[t=15] EVENT:START PID 16

[t=16] Proceso terminado: PID 16 - Finalizado normalmente

[t=16] EVENT:END PID 16

[t=16] EVENT:FINISH PID 16

[t=16] Proceso admitido a READY: PID 17

[t=16] Proceso admitido a READY: PID 18

[t=16] Proceso admitido a READY: PID 19

[t=16] Proceso admitido a READY: PID 20

[t=16] Proceso admitido a READY: PID 21

[t=16] Planificador activo: Prioridad

[t=16] Planificador activo: Prioridad

[t=16] Planificador activo: Prioridad

[t=16] Planificador activo: Prioridad

[t=16] Prioridad ejecuta PID 17

[t=16] EVENT:DISPATCH PID 17

[t=16] EVENT:START PID 17

[t=17] Prioridad devuelve PID 17 a READY

[t=17] EVENT:END PID 17

[t=17] EVENT:PREEMPT PID 17

[t=17] Prioridad ejecuta PID 17

[t=17] EVENT:DISPATCH PID 17

[t=17] EVENT:START PID 17

[t=18] Prioridad devuelve PID 17 a READY

[t=18] EVENT:END PID 17

[t=18] EVENT:PREEMPT PID 17

[t=18] Prioridad ejecuta PID 17

[t=18] EVENT:DISPATCH PID 17

[t=18] EVENT:START PID 17

[t=19] Prioridad devuelve PID 17 a READY

[t=19] EVENT:END PID 17

[t=19] EVENT:PREEMPT PID 17

[t=19] Prioridad ejecuta PID 17

[t=19] EVENT:DISPATCH PID 17

[t=19] EVENT:START PID 17

[t=20] Prioridad devuelve PID 17 a READY

[t=20] EVENT:END PID 17

[t=20] EVENT:PREEMPT PID 17

[t=20] Prioridad ejecuta PID 17

[t=20] EVENT:DISPATCH PID 17

[t=20] EVENT:START PID 17

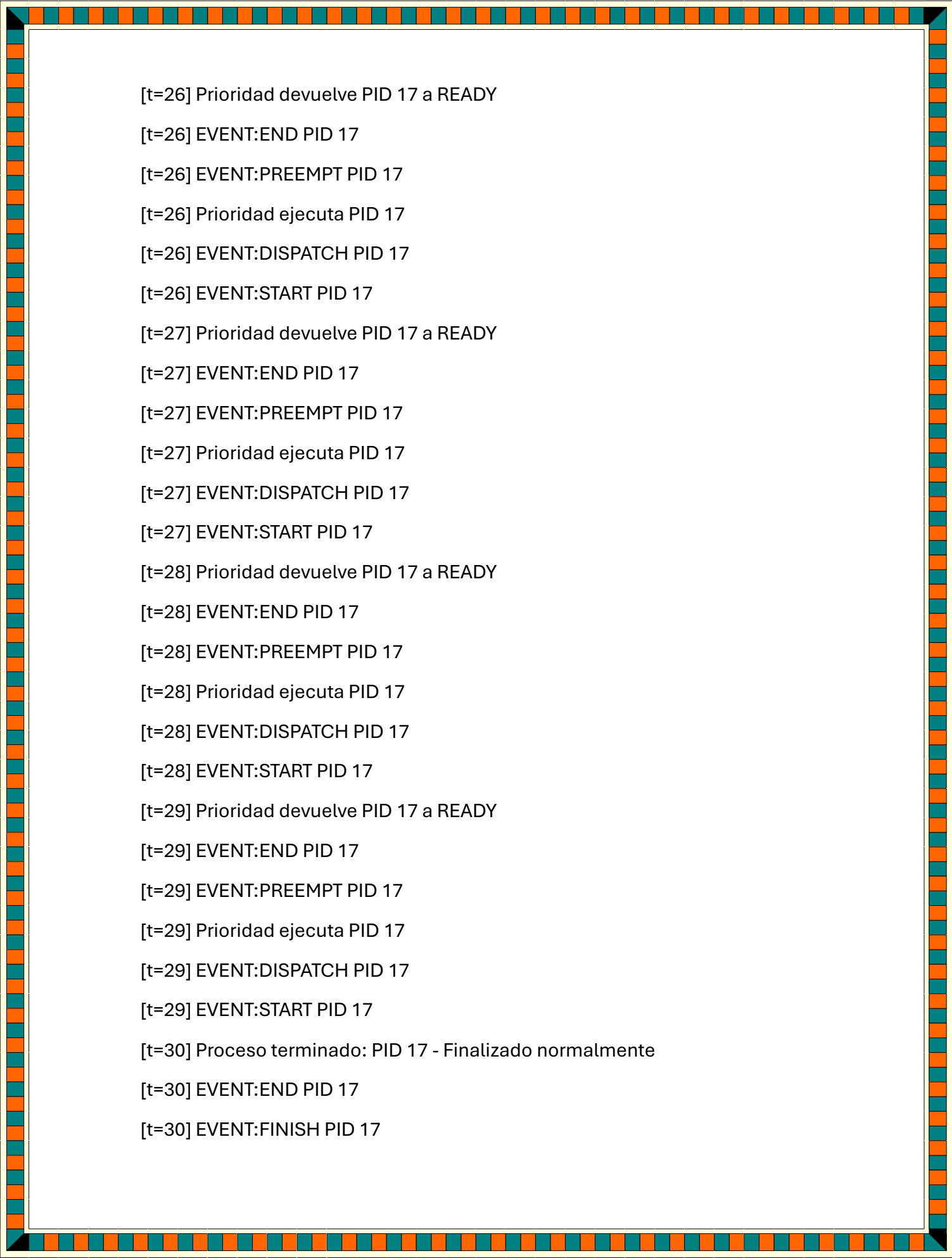
[t=21] Prioridad devuelve PID 17 a READY

[t=21] EVENT:END PID 17

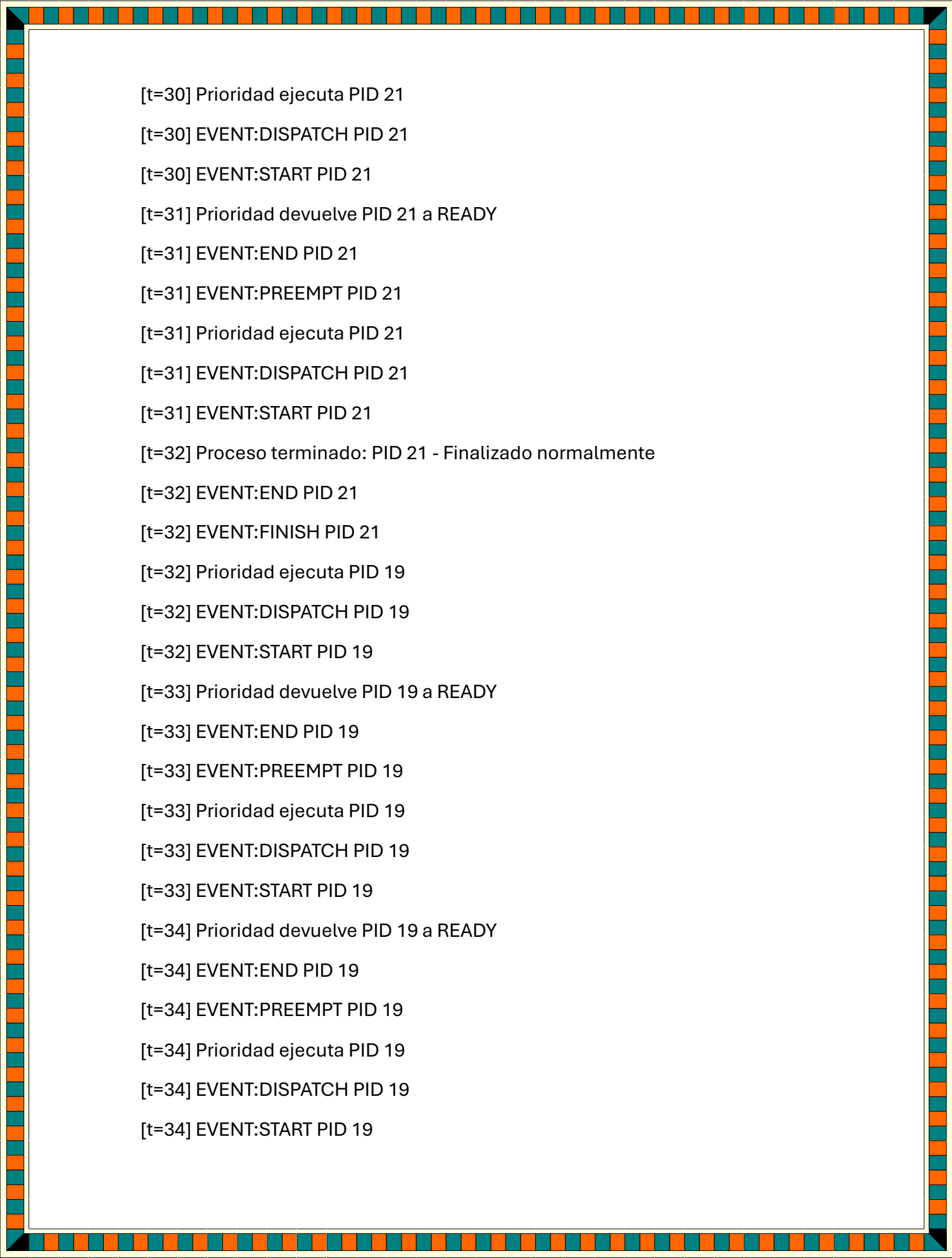
[t=21] EVENT:PREEMPT PID 17



[t=21] Prioridad ejecuta PID 17  
[t=21] EVENT:DISPATCH PID 17  
[t=21] EVENT:START PID 17  
[t=22] Prioridad devuelve PID 17 a READY  
[t=22] EVENT:END PID 17  
[t=22] EVENT:PREEMPT PID 17  
[t=22] Prioridad ejecuta PID 17  
[t=22] EVENT:DISPATCH PID 17  
[t=22] EVENT:START PID 17  
[t=23] Prioridad devuelve PID 17 a READY  
[t=23] EVENT:END PID 17  
[t=23] EVENT:PREEMPT PID 17  
[t=23] Prioridad ejecuta PID 17  
[t=23] EVENT:DISPATCH PID 17  
[t=23] EVENT:START PID 17  
[t=24] Prioridad devuelve PID 17 a READY  
[t=24] EVENT:END PID 17  
[t=24] EVENT:PREEMPT PID 17  
[t=24] Prioridad ejecuta PID 17  
[t=24] EVENT:DISPATCH PID 17  
[t=24] EVENT:START PID 17  
[t=25] Prioridad devuelve PID 17 a READY  
[t=25] EVENT:END PID 17  
[t=25] EVENT:PREEMPT PID 17  
[t=25] Prioridad ejecuta PID 17  
[t=25] EVENT:DISPATCH PID 17  
[t=25] EVENT:START PID 17



[t=26] Prioridad devuelve PID 17 a READY  
[t=26] EVENT:END PID 17  
[t=26] EVENT:PREEMPT PID 17  
[t=26] Prioridad ejecuta PID 17  
[t=26] EVENT:DISPATCH PID 17  
[t=26] EVENT:START PID 17  
[t=27] Prioridad devuelve PID 17 a READY  
[t=27] EVENT:END PID 17  
[t=27] EVENT:PREEMPT PID 17  
[t=27] Prioridad ejecuta PID 17  
[t=27] EVENT:DISPATCH PID 17  
[t=27] EVENT:START PID 17  
[t=28] Prioridad devuelve PID 17 a READY  
[t=28] EVENT:END PID 17  
[t=28] EVENT:PREEMPT PID 17  
[t=28] Prioridad ejecuta PID 17  
[t=28] EVENT:DISPATCH PID 17  
[t=28] EVENT:START PID 17  
[t=29] Prioridad devuelve PID 17 a READY  
[t=29] EVENT:END PID 17  
[t=29] EVENT:PREEMPT PID 17  
[t=29] Prioridad ejecuta PID 17  
[t=29] EVENT:DISPATCH PID 17  
[t=29] EVENT:START PID 17  
[t=30] Proceso terminado: PID 17 - Finalizado normalmente  
[t=30] EVENT:END PID 17  
[t=30] EVENT:FINISH PID 17



[t=30] Prioridad ejecuta PID 21  
[t=30] EVENT:DISPATCH PID 21  
[t=30] EVENT:START PID 21  
[t=31] Prioridad devuelve PID 21 a READY  
[t=31] EVENT:END PID 21  
[t=31] EVENT:PREEMPT PID 21  
[t=31] Prioridad ejecuta PID 21  
[t=31] EVENT:DISPATCH PID 21  
[t=31] EVENT:START PID 21  
[t=32] Proceso terminado: PID 21 - Finalizado normalmente  
[t=32] EVENT:END PID 21  
[t=32] EVENT:FINISH PID 21  
[t=32] Prioridad ejecuta PID 19  
[t=32] EVENT:DISPATCH PID 19  
[t=32] EVENT:START PID 19  
[t=33] Prioridad devuelve PID 19 a READY  
[t=33] EVENT:END PID 19  
[t=33] EVENT:PREEMPT PID 19  
[t=33] Prioridad ejecuta PID 19  
[t=33] EVENT:DISPATCH PID 19  
[t=33] EVENT:START PID 19  
[t=34] Prioridad devuelve PID 19 a READY  
[t=34] EVENT:END PID 19  
[t=34] EVENT:PREEMPT PID 19  
[t=34] Prioridad ejecuta PID 19  
[t=34] EVENT:DISPATCH PID 19  
[t=34] EVENT:START PID 19

[t=35] Prioridad devuelve PID 19 a READY

[t=35] EVENT:END PID 19

[t=35] EVENT:PREEMPT PID 19

[t=35] Prioridad ejecuta PID 19

[t=35] EVENT:DISPATCH PID 19

[t=35] EVENT:START PID 19

[t=36] Prioridad devuelve PID 19 a READY

[t=36] EVENT:END PID 19

[t=36] EVENT:PREEMPT PID 19

[t=36] Prioridad ejecuta PID 19

[t=36] EVENT:DISPATCH PID 19

[t=36] EVENT:START PID 19

[t=37] Prioridad devuelve PID 19 a READY

[t=37] EVENT:END PID 19

[t=37] EVENT:PREEMPT PID 19

[t=37] Prioridad ejecuta PID 19

[t=37] EVENT:DISPATCH PID 19

[t=37] EVENT:START PID 19

[t=38] Prioridad devuelve PID 19 a READY

[t=38] EVENT:END PID 19

[t=38] EVENT:PREEMPT PID 19

[t=38] Prioridad ejecuta PID 19

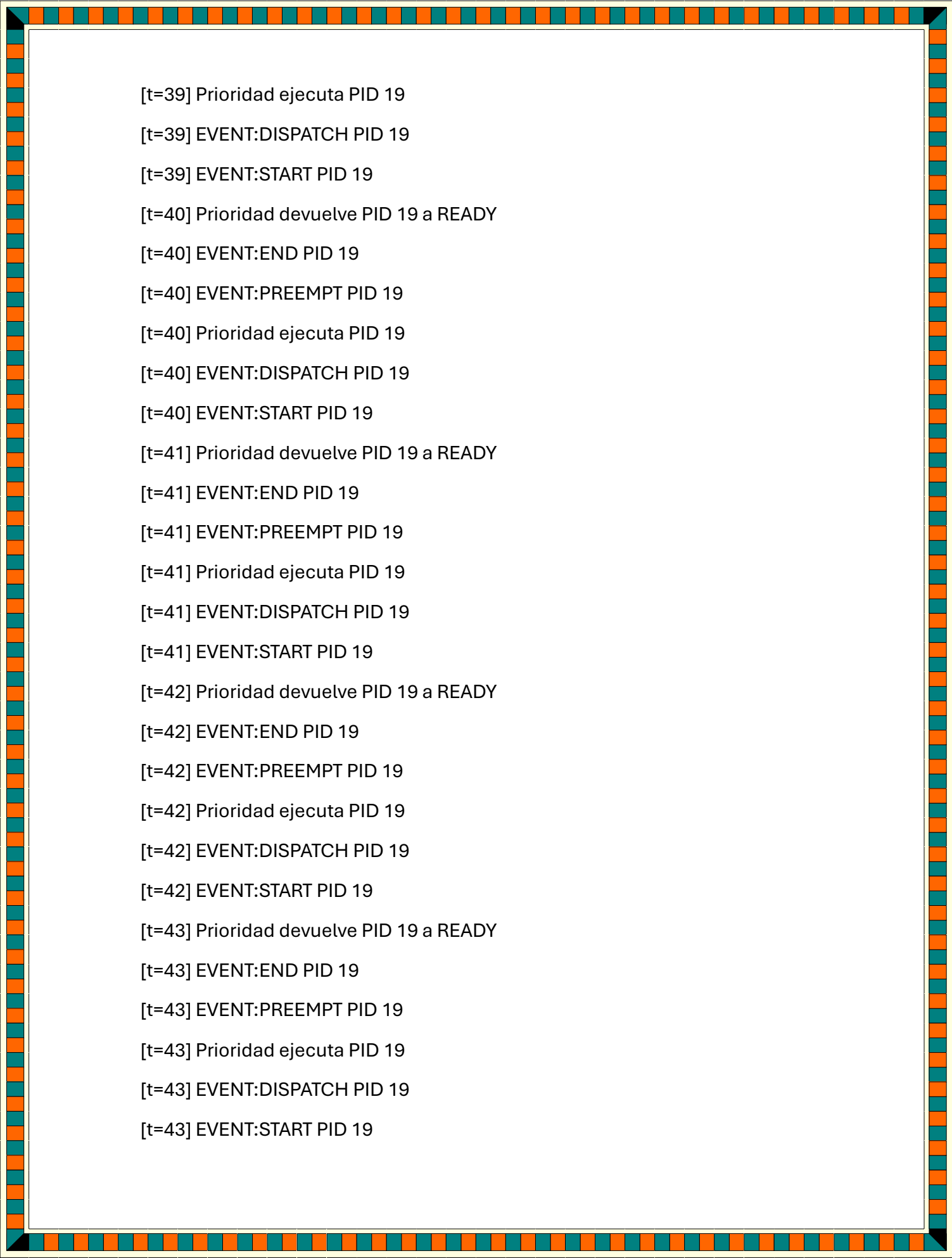
[t=38] EVENT:DISPATCH PID 19

[t=38] EVENT:START PID 19

[t=39] Prioridad devuelve PID 19 a READY

[t=39] EVENT:END PID 19

[t=39] EVENT:PREEMPT PID 19



[t=39] Prioridad ejecuta PID 19  
[t=39] EVENT:DISPATCH PID 19  
[t=39] EVENT:START PID 19  
[t=40] Prioridad devuelve PID 19 a READY  
[t=40] EVENT:END PID 19  
[t=40] EVENT:PREEMPT PID 19  
[t=40] Prioridad ejecuta PID 19  
[t=40] EVENT:DISPATCH PID 19  
[t=40] EVENT:START PID 19  
[t=41] Prioridad devuelve PID 19 a READY  
[t=41] EVENT:END PID 19  
[t=41] EVENT:PREEMPT PID 19  
[t=41] Prioridad ejecuta PID 19  
[t=41] EVENT:DISPATCH PID 19  
[t=41] EVENT:START PID 19  
[t=42] Prioridad devuelve PID 19 a READY  
[t=42] EVENT:END PID 19  
[t=42] EVENT:PREEMPT PID 19  
[t=42] Prioridad ejecuta PID 19  
[t=42] EVENT:DISPATCH PID 19  
[t=42] EVENT:START PID 19  
[t=43] Prioridad devuelve PID 19 a READY  
[t=43] EVENT:END PID 19  
[t=43] EVENT:PREEMPT PID 19  
[t=43] Prioridad ejecuta PID 19  
[t=43] EVENT:DISPATCH PID 19  
[t=43] EVENT:START PID 19



[t=44] Prioridad devuelve PID 19 a READY  
[t=44] EVENT:END PID 19  
[t=44] EVENT:PREEMPT PID 19  
[t=44] Prioridad ejecuta PID 19  
[t=44] EVENT:DISPATCH PID 19  
[t=44] EVENT:START PID 19  
[t=45] Proceso terminado: PID 19 - Finalizado normalmente  
[t=45] EVENT:END PID 19  
[t=45] EVENT:FINISH PID 19  
[t=45] Prioridad ejecuta PID 20  
[t=45] EVENT:DISPATCH PID 20  
[t=45] EVENT:START PID 20  
[t=46] Prioridad devuelve PID 20 a READY  
[t=46] EVENT:END PID 20  
[t=46] EVENT:PREEMPT PID 20  
[t=46] Prioridad ejecuta PID 20  
[t=46] EVENT:DISPATCH PID 20  
[t=46] EVENT:START PID 20  
[t=47] Prioridad devuelve PID 20 a READY  
[t=47] EVENT:END PID 20  
[t=47] EVENT:PREEMPT PID 20  
[t=47] Prioridad ejecuta PID 20  
[t=47] EVENT:DISPATCH PID 20  
[t=47] EVENT:START PID 20  
[t=48] Prioridad devuelve PID 20 a READY  
[t=48] EVENT:END PID 20  
[t=48] EVENT:PREEMPT PID 20

[t=48] Prioridad ejecuta PID 20

[t=48] EVENT:DISPATCH PID 20

[t=48] EVENT:START PID 20

[t=49] Proceso terminado: PID 20 - Finalizado normalmente

[t=49] EVENT:END PID 20

[t=49] EVENT:FINISH PID 20

[t=49] Prioridad ejecuta PID 18

[t=49] EVENT:DISPATCH PID 18

[t=49] EVENT:START PID 18

[t=50] Proceso terminado: PID 18 - Finalizado normalmente

[t=50] EVENT:END PID 18

[t=50] EVENT:FINISH PID

Escenario Productor-Consumidor:

Simulador de Gestor de Procesos

Dashboard Configuración Ejecución Sincronización Línea de tiempo Comparación Historial

Escenario Productor-Consumidor (memoria compartida + mutex + semaforos)

PID Productor: 1001  
PID Consumidor: 2001  
Dato a producir: dato-demo

Producir Consumir Demo Productor-Consumidor Actualizar estado

Prods: 2 Cons: 2 Ciclos: 3 Buffer: 1 Ejecutar escenario automatico

Paso 2: el productor toma el mutex, escribe y libera un dato

**Productores produjeron y notificaron a los consumidores**

Productor PID 8001 Memoria compartida [dato-demo-8001] Consumidor PID 9001

Mutex: libre | empty=0 | filled=1  
Espera mutex=[] | empty=[8002] | filled=[9002]

Estado actual de sincronizacion

Comunicacion (SharedMemory)  
- Valor actual: [dato-demo-8001]

Sincronizacion (Mutex)  
- Bloqueado: false  
- Propietario: ninguno  
- Cola espera mutex: []

Semaforo empty

Listo para simular procesos.

Log final de la demo:

Comunicacion (SharedMemory)

- Valor actual: []

Sincronizacion (Mutex)

- Bloqueado: false

- Propietario: ninguno

- Cola espera mutex: []

Semaforo empty

- Valor: 1

- Esperando: []

Semaforo filled

- Valor: 0

- Esperando: [9002]

## Conclusiones

Aciertos y aprendizajes

- La separación por capas (modelo / servicios / controlador / vista) facilita el mantenimiento y la extensión del simulador.
- Emitir eventos estructurados es esencial para construir visualizaciones temporales (Gantt) útiles y comprensibles.
- La simulación del productor-consumidor con buffer configurable permite observar efectos didácticos de capacidad, bloqueo y notificación.

Dificultades

- Fue necesario ajustar la estrategia de renderizado de la línea de tiempo y los componentes gráficos para evitar incompatibilidades en distintos entornos gráficos.

- Para detectar deadlocks complejos haría falta implementar un análisis de wait-for graph; actualmente se usa una heurística simple que detecta si todos los procesos activos están en espera.

#### Trabajo futuro

- Mejorar detección y resolución de deadlocks (algoritmo de detección por grafo y políticas de recuperación).
  - Añadir demo de comunicación por paso de mensajes como alternativa a memoria compartida.
- Generar reportes automáticos (PDF) con capturas y checklist integrados.
  - Incluir métricas de cobertura de pruebas y pipeline de CI que ejecute tests automáticamente.

## Referencias

- Andrew S. Tanenbaum. Modern Operating Systems.
- Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts.
- Documentación de JavaFX para interfaces gráficas.
- Documentación de prácticas sobre semáforos y exclusión mutua en sistemas operativos.